

流水线模拟器实验报告

一、实验目的

- (1) 加深对计算机流水线基本概念的理解;
- (2) 理解 MIPS 结构如何用 5 段流水线来实现, 理解各段的功能和基本操作;
- (3) 加深对数据冲突、结构冲突、控制冲突的理解, 并能够分析这些冲突对 CPU 性能的影响;
- (4) 进一步理解解决数据冲突的方法, 掌握如何应用定性技术来减少数据冲突引起的流水线停顿。

二、实验任务

设计一个流水线模拟器\寻找开源流水线模拟器, 模拟MIPS的五段流水线, 并支持图形交互, 单步执行、执行到程序结束, 各个段、寄存器状态的查看, 提供是否使用定向路径的功能选项, 支持load、store、add、beqz操作、执行的程序可以通过直接输入。

并设计三种不同的代码组合, 实现三种场景 (分别是无冲突、RAW冲突、分支跳转) 的演示, 并对三种场景进行分析。

三、模拟器设计思想

在本次实验中, 选择自己设计一个流水线模拟器。

首先分析要做的任务: 使用图形界面模拟MIPS五段流水线的执行过程。

为此, 模拟器首先要能够读取并处理文本成指令格式, 并维护一个指令列表。然后, 为了模拟流水线功能, 需要维护五个段, 并处理指令在段间推入的相关逻辑, 中间涉及到定向/非定向对RAW冲突的处理, 以及跳转指令的相关逻辑实现, 最后, 通过流水线时空图动态展示。

因此设计:

Pipeline类: 实现五段流水线核心逻辑

Instruction类: 表示指令并记录指令阶段历史

RegisterFile类: 模拟寄存器

Memory类: 模拟内存

PipelineGUI类: 负责用户交互、数据显示等。

Instruction类

功能: 表示一条汇编指令, 负责解析指令文本, 记录指令的流水线阶段历史。

```

class Instruction:
    def __init__(self, text, idx):
        self.text = text.strip() # 指令文本内容
        self.idx = idx # 指令在程序中的索引
        self.stage_history = [] # 记录每周期所处阶段
        self.parse() # 解析指令
        self.finished = False # 指令是否已完成
        self.entered_if = False

    def parse(self):
        if self.text.endswith(':'):
            self.is_label = True
            self.op = "label" # 标签
            self.args = []
        else:
            self.is_label = False
            parts = self.text.replace(',', ' ').split() # 按空格分割, 去掉逗号
            self.op = parts[0] # 操作码
            self.args = parts[1:] # 操作数参数

```

text是读入的指令行文本，通过parse方法，将操作码，和操作数读入到op、args中。

idx是实现指令跳转的关键属性，但值得一提的是，在本次模拟器实现中，idx并不是严格模拟的指令地址。

对于label行，我并没有在指令读取阶段将其idx跳过，而是将其作为一条伪指令，使用is_label属性标识，一起存入了指令列表中，方便后续跳转时直接遍历指令列表查找。

stage_history则是记录了每个周期指令所处的段，IF、ID..或插入气泡的记录。

finished属性用来记录指令是否完成，判断结束条件会用到(否则无法实现执行到结束功能)。

entered_if用来记录指令是否已经进入流水线。

RegisterFile类和Memory类

```
class RegisterFile:
    def __init__(self):
        self.regs = [0] * 32 # 32个寄存器，初始值为0

    def __getitem__(self, reg):
        idx = int(reg.replace('$', '')) # 获取寄存器编号
        return self.regs[idx] # 返回寄存器值

    def __setitem__(self, reg, value):
        idx = int(reg.replace('$', '')) # 获取寄存器编号
        self.regs[idx] = value # 设置寄存器值

class Memory:
    def __init__(self):
        self.mem = {} # 内存用字典表示，地址为key，值为value

    def __getitem__(self, addr):
        return self.mem.get(addr, 0) # 读取内存，默认值为0

    def __setitem__(self, addr, value):
        self.mem[addr] = value # 写内存
```

不过多介绍，实现简单且在本次模拟中并不重要。

PipeLine类以及关键功能思路

功能：实现MIPS五段流水线的核心逻辑，包括指令推进、数据冒险检测、分支处理、数据前递等。

主要属性

```
def __init__(self, regfile, memory, use_forwarding):
    self.regfile = regfile # 寄存器文件对象
    self.memory = memory # 内存对象
    self.use_forwarding = use_forwarding # 是否启用数据前递
    self.instructions = [] # 指令列表
    self.pc = 0 # 程序计数器
    self.cycle = 0 # 当前周期数
    self.finished = False # 是否执行完毕
    self.stages = [None] * 5 # 五个流水线阶段: [IF, ID, EX, MEM, WB]
    self.in_pipeline = [] # 只记录已进流水线的指令
    self.branch_pending = False # 是否有分支待处理
    self.branch_target = None # 分支目标指令索引
    self.stall = 0 # 暂停周期数（未实际使用）
```

其中self.stages用于模拟五个段。

load_program、execute、memory_access、write_back方法

```
def load_program(self, instructions):
    self.instructions = instructions # 加载指令
    self.pc = 0 # 重置PC
    self.cycle = 0 # 重置周期
    self.finished = False # 重置完成标志
    self.stages = [None] * 5 # 清空流水线
    self.in_pipeline = [] # 清空流水线指令
    self.branch_pending = False # 清空分支标志
    self.branch_target = None # 清空分支目标
    self.stall = 0 # 清空暂停
    for inst in self.instructions:
        inst.stage_history = [] # 清空每条指令的阶段历史
        inst.finished = False # 重置完成标志
        inst.entered_if = False # 重置IF标志
```

load_program方法负责加载指令到当前的PipeLine，而调用这个方法的位置则是在PipelineGUI类。

```

def execute(self, inst):
    # 执行阶段：计算结果或地址
    if inst.op == "add":
        rd, rs, rt = inst.args # 目的寄存器、两个源寄存器
        inst.result = self.regfile[rs] + self.regfile[rt] # 计算加法
    elif inst.op == "lw":
        rt, offset_rs = inst.args # 目标寄存器，偏移量和基址
        offset, rs = offset_rs.replace(')', '').split('(')
        addr = self.regfile[rs] + int(offset) # 计算内存地址
        inst.mem_addr = addr # 保存地址
    elif inst.op == "sw":
        rt, offset_rs = inst.args # 源寄存器，偏移量和基址
        offset, rs = offset_rs.replace(')', '').split('(')
        addr = self.regfile[rs] + int(offset)
        inst.mem_addr = addr
    # beqz在step里处理分支冒险

def memory_access(self, inst):
    # 访存阶段
    if inst.op == "lw":
        inst.result = self.memory[inst.mem_addr] # 读取内存
    elif inst.op == "sw":
        self.memory[inst.mem_addr] = self.regfile[inst.args[0]] # 写内存

def write_back(self, inst):
    # 写回阶段
    if inst.op == "add":
        self.regfile[inst.args[0]] = inst.result # 写回寄存器
    elif inst.op == "lw":
        self.regfile[inst.args[0]] = inst.result # 写回寄存器

```

execute方法、memory_access方法、write_back方法分别在指令在EX段、MEM段、WB段时调用。EX段，load和store指令将指令中指定的内容与偏移量相加，形成有效访存地址；add指令对从寄存器中读取的数据进行计算；beqz指令的EX段处理则在step方法中实现。MEM段，load指令从有效地址指向的存储器单元将数据结果读取出来，store指令将数据写入这个有效地址指向的存储器单元。WB段，add指令和lw指令将结果写回寄存器。

step方法

step方法是本次模拟中最关键的部分。涉及到指令推进，RAW冲突，分支跳转逻辑。

step方法是本次模拟的核心推进函数，每调用一次就模拟流水线推进一个时钟周期，它负责：

- 检查并处理数据冒险和分支冒险

- 推进流水线各阶段
- 执行指令、访存、写回
- 记录每条指令的流水线阶段历史
- 判断程序是否执行完毕

1. 检查是否已经执行完毕

```
if self.finished:
    return
```

如果流水线已经全部执行完毕，则直接返回，不再推进。

2. 非定向时检查ID段是否有数据冒险

```
stall_needed = False
if not self.use_forwarding and self.stages[1]:
    id_inst = self.stages[1]
    id_srcs = []
    # 根据指令类型，提取源寄存器
    if id_inst.op == "add":
        id_srcs = [id_inst.args[1], id_inst.args[2]]
    elif id_inst.op == "lw":
        id_srcs = [id_inst.args[1].split('(')[1][:-1]]
    elif id_inst.op == "sw":
        id_srcs = [id_inst.args[0], id_inst.args[1].split('(')[1][:-1]]
    elif id_inst.op == "beqz":
        id_srcs = [id_inst.args[0]]
    # 检查EX、MEM、WB段是否有未写回的相关寄存器
    for stage_idx in [2, 3, 4]:
        inst = self.stages[stage_idx]
        if inst and inst.op in ("add", "lw"):
            dest = inst.args[0]
            if dest in id_srcs:
                if stage_idx != 4:
                    stall_needed = True
                    break
```

解释：

如果没有数据前递（use_forwarding为False），检查数据冒险。对ID段的指令，提取其所有源寄存器。接着检查EX、MEM、WB段是否有未写回的相关目的寄存器，如果有且还没写回（即不在WB段），则需要插入 stall，暂停流水线推进。该处目前只是检查是否需要插入stall，并作标记。

当然，WAR和WAW冒险在顺序流水线是不存在的，我们不用考虑。

3. 检查定向下的特殊冒险

```
forwarding_stall_needed = False
if self.use_forwarding and self.stages[1]:
    ex_inst = self.stages[1]
    ex_srcs = []
    # 提取源寄存器
    if ex_inst.op == "add":
        ex_srcs = [ex_inst.args[1], ex_inst.args[2]]
    elif ex_inst.op == "lw":
        ex_srcs = [ex_inst.args[1].split('(')[1][:-1]]
    elif ex_inst.op == "sw":
        ex_srcs = [ex_inst.args[0], ex_inst.args[1].split('(')[1][:-1]]
    elif ex_inst.op == "beqz":
        ex_srcs = [ex_inst.args[0]]
    inst = self.stages[2]
    if inst and inst.op == "lw":
        dest = inst.args[0]
        if dest in ex_srcs:
            forwarding_stall_needed = True
```

解释：

定向技术实现相当复杂，而本次模拟只是个简单实现，并没有对每个部件都进行模拟，只是想正确模拟出结果。

那么思考：本次实验只实现了lw，add，beqz，sw指令。其中RAW冲突中的W，不会是SW和beqz指令，我们只需考虑add和lw指令即可。假设一个只存在add指令的程序。

```
add $t0, $t1, $t2    # 指令A: 结果在EX阶段结束时可用（EX/MEM寄存器）
add $t3, $t0, $t4    # 指令B: 需在EX阶段使用$t0的值
```

add指令在EX阶段结束时生成，并存储到EX/MEM流水线寄存器中。

定向技术可将EX/MEM寄存器中的值直接转发到后续指令的EX阶段输入，无需等到WB阶段写入寄存器文件。

而add指令的RAW冒险仅会发生在相邻指令之间，且结果在EX阶段即可通过旁路传递。那我们就能发现这么一个结果：

对于仅含add指令的程序，定向技术可以完美覆盖所有的数据冒险！

因此，我们便可转换模拟思路，仅处理lw和add指令间可能存在的数据冒险即可。

```
lw  $t0, 0($s0)    # 结果在MEM阶段结束时才可用
add $t1, $t0, $s1  # 需要$t0的值（EX阶段）
```

lw的结果在MEM阶段结束前不可用，而后续add的EX阶段需要该值。

定向技术无法覆盖这种情况，需要插入一个周期的阻塞。

因此，我们检查这样的情况：对于即将进入EX阶段的指令，提取它的源寄存器，在检查之前的指令中的LW指令，是否有未写回的相关目的寄存器，如果有则记录，并在后续进行相关插入stall和暂停处理。

其余情况可直接通过前递解决，不需处理。

4. 检查IF段是否需要暂停（分支指令beqz）

```
beqz_IF_pause = False
if self.stages[0] and self.stages[0].op == "beqz":
    beqz_IF_pause = True
```

解释：

- 如果IF段是 beqz 分支指令，则暂停取指，等待分支判断结果。此处是作标记。

5. 判断分支跳转并预处理

```
if self.stages[1] and self.stages[1].op == "beqz":
    rs, label = self.stages[1].args
    if self.regfile[rs] == 0:
        for i, ins in enumerate(self.instructions):
            if ins.text.startswith(label + ":"):
                self.branch_pending = True
                self.branch_target = i+1
                break
    else:
        self.branch_pending = False
```

解释：

在ID段遇到 beqz ，在EX阶段判断分支是否成立。beqz的源操作数args中存的是label，遍历指令列表找到label的位置。由于label在指令列表中是当作一条伪指令存入的，找到label后需要将idx+1，即是目标指令的位置。如果分支成立，设置分支待处理标志和目标指令索引。

6. 推进流水线各阶段（后段先推进）

```
self.stages[4] = self.stages[3] # MEM->WB
self.stages[3] = self.stages[2] # EX->MEM
```

解释：

- 先将MEM段的指令推进到WB段，EX段推进到MEM段。

7. 处理ID/EX推进和写入（根据是否需要stall）

数据前递下的stall

```
if self.use_forwarding and self.stages[1]:
    if forwarding_stall_needed:
        self.stages[1].stage_history.append('ID')
    else:
        if self.stages[1].stage_history[-1] == 'ID':
            self.stages[1].stage_history.append('stall')
        else:
            self.stages[1].stage_history.append('ID')
```

解释：

- 如果需要前递stall，ID段指令停留在ID阶段，否则正常推进。

stall或正常推进

```
if forwarding_stall_needed:
    self.stages[2] = None
    if self.stages[0] and self.stages[0].entered_if:
        self.stages[0].stage_history.append('stall')
elif stall_needed:
    self.stages[2] = None
    if self.stages[1]:
        self.stages[1].stage_history.append('stall')
    if self.stages[0] and self.stages[0].entered_if:
        self.stages[0].stage_history.append('stall')
else:
    self.stages[2] = self.stages[1]
    if self.stages[1] and not self.use_forwarding:
        self.stages[1].stage_history.append('ID')
    self.stages[1] = self.stages[0]
    # IF段推进
    if self.branch_pending:
        if self.branch_target is not None:
            inst = self.instructions[self.branch_target]
            while len(inst.stage_history) < self.cycle:
                inst.stage_history.append("")
            if not inst.entered_if:
                inst.stage_history.append('IF')
                inst.entered_if = True
                self.in_pipeline.append(inst)
            self.stages[0] = inst
            self.pc = self.branch_target + 1
        else:
            self.stages[0] = None
            self.branch_pending = False
            self.branch_target = None
    else:
        if self.pc < len(self.instructions) and not beqz_IF_pause:
            inst = self.instructions[self.pc]
            while inst.is_label:
                self.pc += 1
            inst = self.instructions[self.pc]
            while len(inst.stage_history) < self.cycle:
                inst.stage_history.append("")
            if not inst.entered_if:
                inst.stage_history.append('IF')
                inst.entered_if = True
                self.in_pipeline.append(inst)
            self.stages[0] = inst
```

```

        self.pc += 1
    else:
        self.stages[0] = None

```

解释：

- 如果需要stall，EX段不推进，ID/IF段补 stall。否则正常推进：EX <- ID，ID <- IF，IF取新指令（分支跳转时取目标指令，否则顺序取下一条）。如果分支跳转成立，将pc跳转至目标索引，并重置相关判断。正常推进流程：取pc处指令，检查如果是label，则PC+1，重新读指令，直到读到的指令不是label(label在本次模拟中算一条伪指令)。如果没有进入IF段，就在阶段历史中记录IF，并将entered_IF置为真（这是为了避免暂停前后多次进入IF段的bug额外进行的判断）。

8. 其它流水线阶段推进和写入

```

if self.stages[2]:
    self.stages[2].stage_history.append('EX')
if self.stages[3]:
    self.stages[3].stage_history.append('MEM')
if self.stages[4] and not self.stages[4].finished:
    self.stages[4].stage_history.append('WB')
    self.write_back(self.stages[4])
    self.stages[4].finished = True

```

解释：

- EX、MEM、WB阶段分别记录阶段历史。WB阶段执行写回操作，并标记指令已完成。

9. MEM/EX阶段操作

```

if self.stages[3]:
    self.memory_access(self.stages[3])
if self.stages[2]:
    self.execute(self.stages[2])

```

解释：

MEM阶段执行访存操作（如lw/sw）。EX阶段执行算术运算或地址计算。

10. 周期数加一

```

self.cycle += 1

```

解释：

- 每推进一次，周期数加一。

11. 检查是否全部完成

```
if all(x is None or getattr(x, 'finished', False) for x in self.stages) and self.pc > len(self.stages) - 1:
    self.finished = True
if all(x is None or getattr(x, 'finished', False) for x in self.stages) and self.pc >= len(self.stages):
    self.pc += 1
```

解释：

- 如果所有流水线段都为空或已完成，且PC已越界，则标记程序执行完毕。之所以这样写，是为了避免最后一个周期提前判断结束导致指令提前一个周期结束。而只判断`pc==len`又会导致无法正确结束。这里并没有分析清楚原因。但是依然成功实现了指令的正确结束判断。

PipelineGUI类

基于tkinter库实现。负责：

- 用户交互
- 数据展示
- 状态展示

由于图形界面并不是本次模拟的重点，因此不做详细的代码解释，在此只大概介绍功能。

主要属性

- `self.root`：Tkinter主窗口对象。
- `self.instructions`：当前加载的指令对象列表。
- `self.regfile`：寄存器文件对象（`RegisterFile` 实例）。
- `self.memory`：内存对象（`Memory` 实例）。
- `self.pipeline`：流水线对象（`Pipeline` 实例），负责核心模拟逻辑。
- `self.code_input`：多行文本框，供用户输入MIPS汇编代码。
- `self.load_btn`、`self.step_btn`、`self.run_btn`：按钮，分别用于加载程序、单步执行、运行到结束。
- `self.forward_var`：布尔变量，表示是否启用数据前递（定向）。
- `self.forward_check`：复选框，切换数据前递模式。
- `self.pipeline_table`：表格控件，显示每条指令在各周期的流水线阶段。
- `self.pipeline_xscroll`：横向滚动条，配合流水线表格使用。
- `self.reg_table`：表格控件，显示32个寄存器的当前值。
- `self.status_label`：标签，显示当前模拟器状态（如“等待加载程序”、“已执行X周期”等）。

主要方法

1. `__init__(self, root)`

- 初始化所有界面控件和数据对象。
- 设置窗口标题、布局、控件属性和事件绑定。
- 初始化寄存器表格。

2. load_program(self)

- 从 code_input 文本框读取用户输入的汇编代码。
- 解析为 Instruction 对象列表，重置寄存器和内存。
- 创建新的 Pipeline 对象，并加载指令。
- 刷新流水线表格和寄存器表格，更新状态栏。

3. step(self)

- 单步推进流水线（调用 Pipeline.step()）。
- 刷新流水线表格和寄存器表格。
- 更新状态栏显示当前已执行周期数。
- 若未加载程序或已结束，弹出警告或提示。

4. run_to_end(self)

- 循环推进流水线直到程序结束。
- 刷新流水线表格和寄存器表格。
- 更新状态栏显示总周期数。
- 若未加载程序，弹出警告。

5. update_pipeline_table(self)

- 清空并重建流水线表格内容。
- 横向显示每条指令在每个周期的流水线阶段（如IF、ID、EX、MEM、WB、stall）。
- 动态调整表格列数和内容，支持横向滚动。

6. update_reg_table(self)

- 清空并重建寄存器表格内容。
- 实时显示32个寄存器的当前值。

7. toggle_forwarding(self)

- 切换流水线的数据前递（定向）模式。
- 由前递复选框触发，直接修改 Pipeline 对象的 use_forwarding 属性。

界面交互流程

1. 用户输入汇编代码，点击“加载程序”。
2. 可选择是否启用“定向(数据前递)”。
3. 点击“单步执行”或“运行到结束”，观察流水线表格和寄存器表格的动态变化。

4. 状态栏实时提示当前状态和周期数。

四、测试代码组合以及执行结果分析

无冲突场景

```
.text
main:
    LW  $r1, 0($r2)
    ADD $r1, $r2, $r3
    SW  $r3, 0($r2)
    ADD $r4, $r3, $r5
.data
    .align 2
```

自设计模拟器运行结果

MIPS五段流水线模拟器

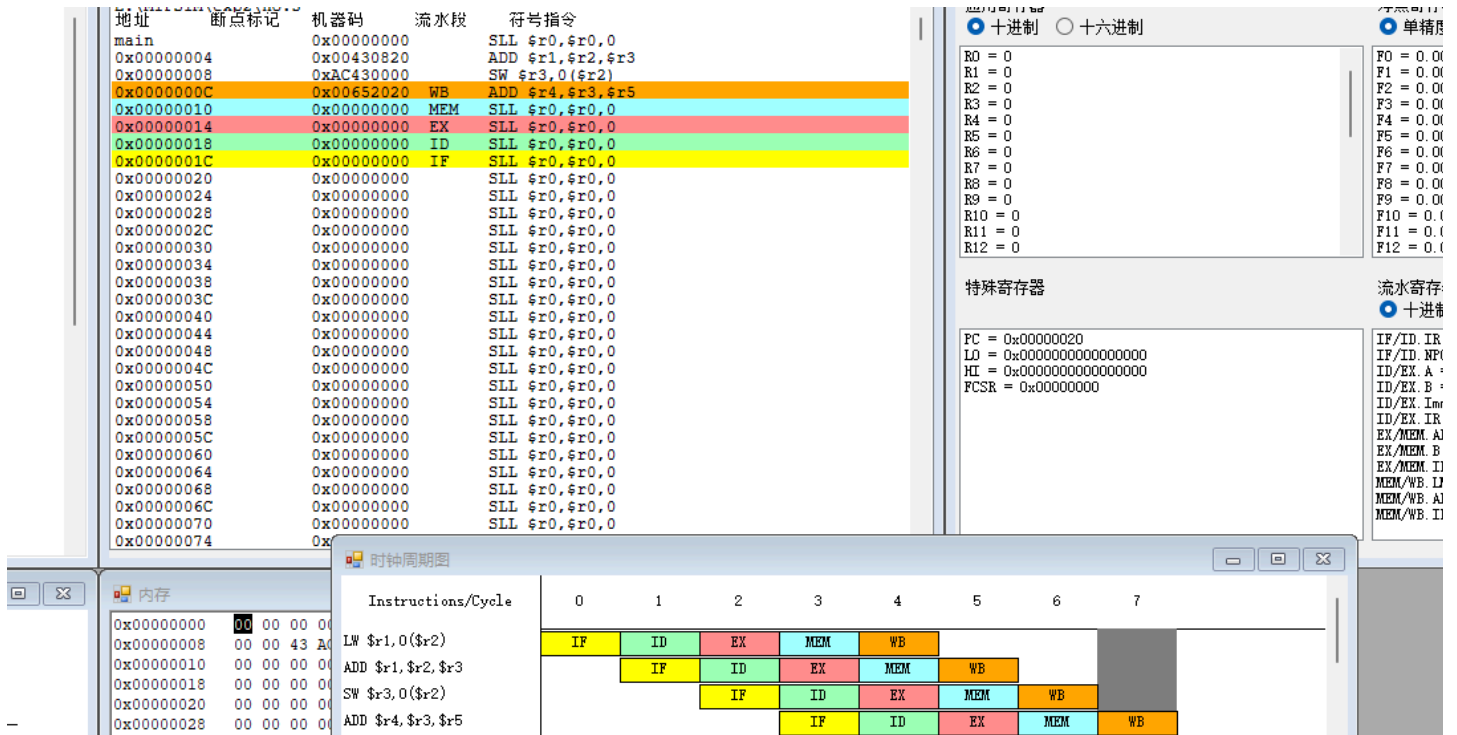
lw \$1, 0(\$2)
add \$1, \$2, \$3
sw \$3, 0(\$2)
add \$4, \$3, \$5

加载程序单步执行运行到结束

指令	C1	C2	C3	C4	C5	C6	C7	C8
lw \$1, 0(\$2)	IF	ID	EX	MEM	WB			
add \$1, \$2, \$3		IF	ID	EX	MEM	WB		
sw \$3, 0(\$2)			IF	ID	EX	MEM	WB	
add \$4, \$3, \$5				IF	ID	EX	MEM	WB

寄存器	值
\$0	0
\$1	0
\$2	0
\$3	0
\$4	0
\$5	0
\$6	0
\$7	0

MIPS流水线模拟器运行结果



发现模拟正确。可以看到没有发生任何冲突，指令依次进入流水线，没有插入任何stall。

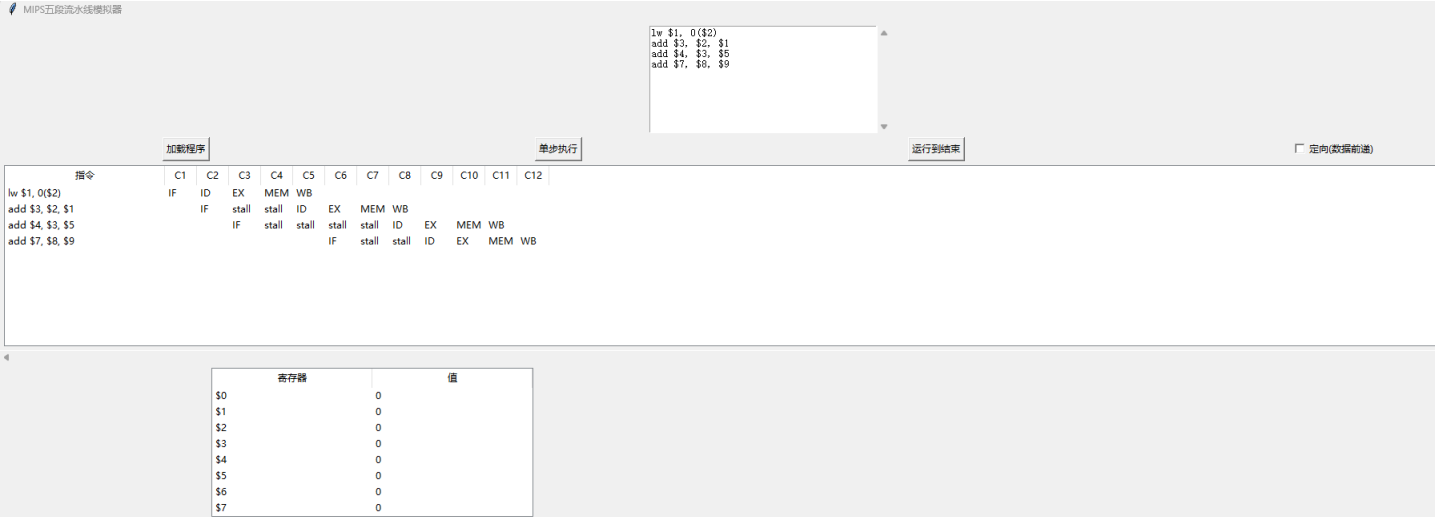
RAW冲突场景

```
.text
main:
    LW $r1, 0($r2)
    ADD $r3, $r2, $r1
    ADD $r4, $r3, $r5
    ADD $r7, $r8, $r9

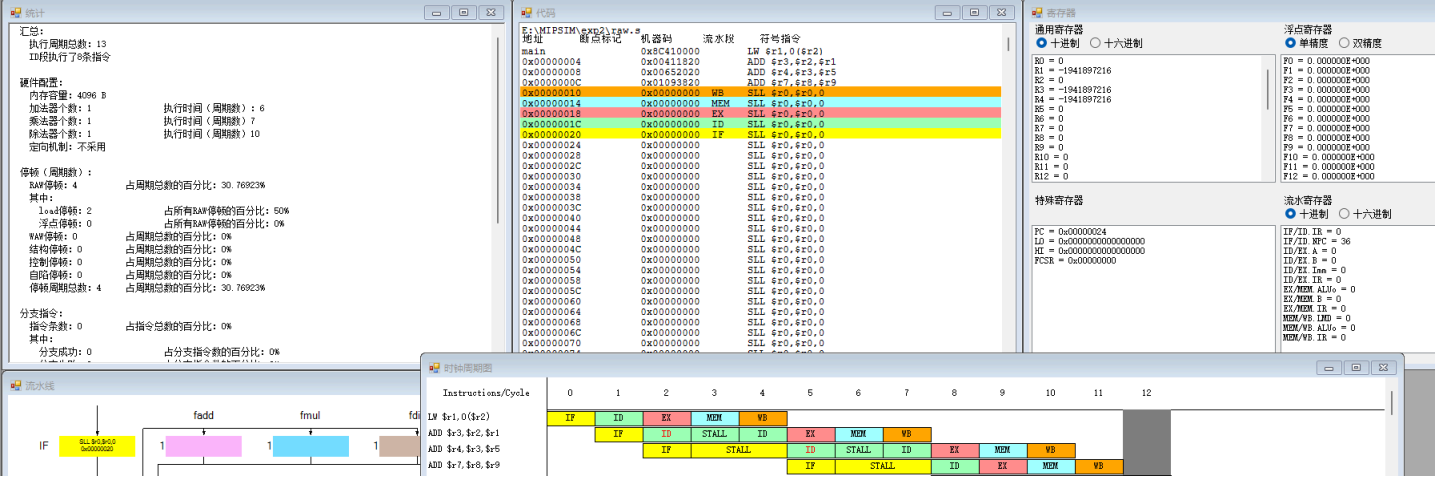
.data
    .align 2
```

非定向

非定向自设计模拟器运行结果



非定向流水线模拟器运行结果



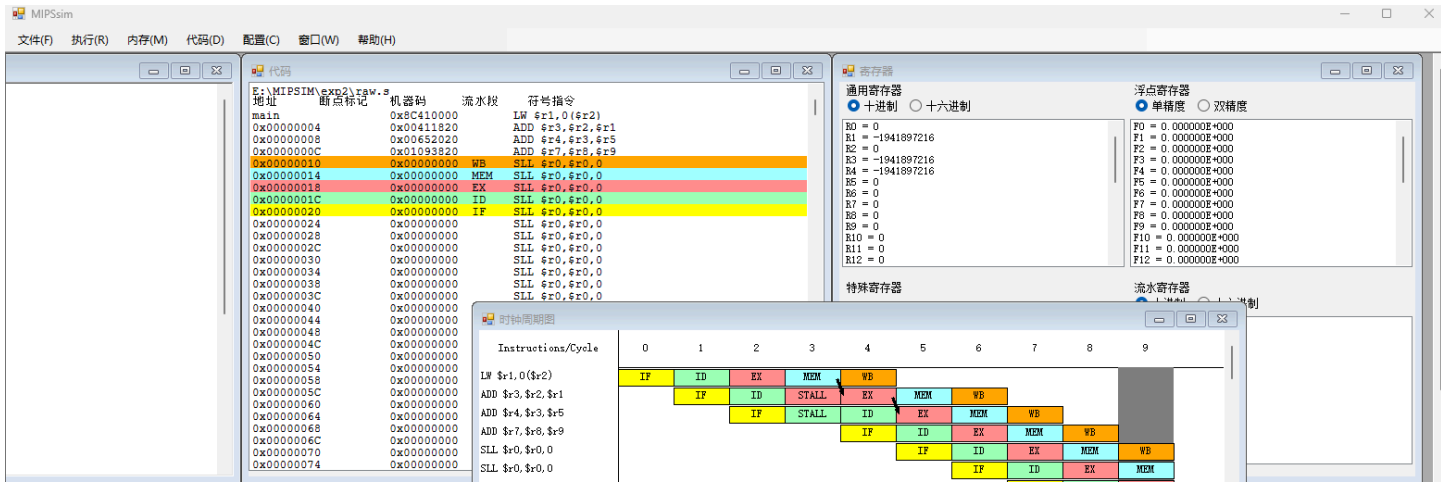
发现模拟正确。分析场景：第二条指令在ID段发现源寄存器\$1是第一条指令的目的寄存器，因此插入stall，直到第一条指令WB写回，同时阻止IF和ID的流入，直到第二条指令ID段结束。因此第四条指令在第六个周期才流入IF段。第三条指令在第六个周期重新尝试进入ID段，检测到冲突，\$3在第二条指令中尚未被写回，因此ID段被延期到了第二条指令的WB周期才流入，而第四条指令因此被阻止流入到ID段。之后正常流水线传递。

定向

定向自设计模拟器运行结果



定向流水线模拟器运行结果

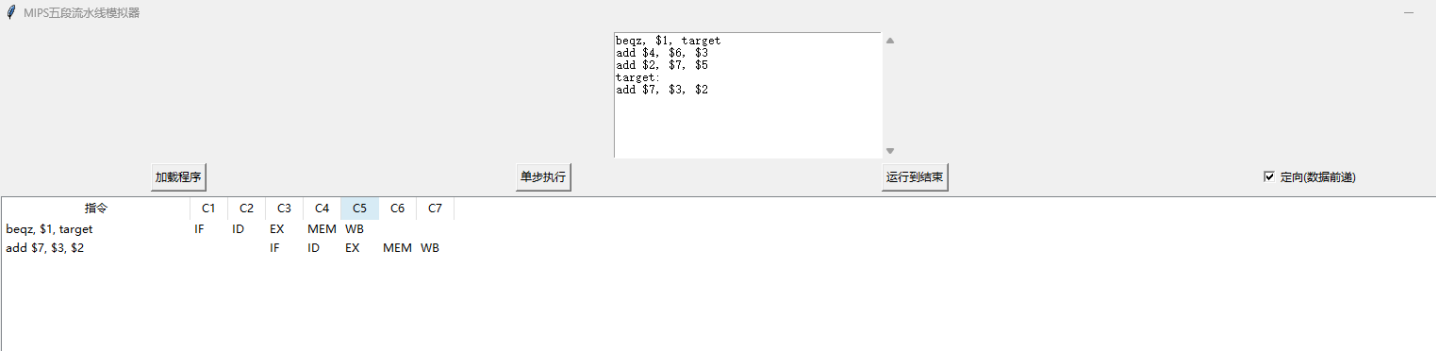


发现模拟正确。分析场景：第二条指令EX段所需要的\$1在lw的MEM周期后才能准备好送到EX段（通过定向技术直接转发到EX段）因此第二条指令的ID后被插入stall等待，同时ID段的流入被阻止，第三条指令被插入stall，在第五个周期才能流入ID段。虽然第三条指令与第二条指令存在\$3的数据依赖，但是通过定向技术将EX/MEM寄存器的值直接转发到EX段，使得不用继续等待直到WB,而是继续顺序执行，直到程序结束。

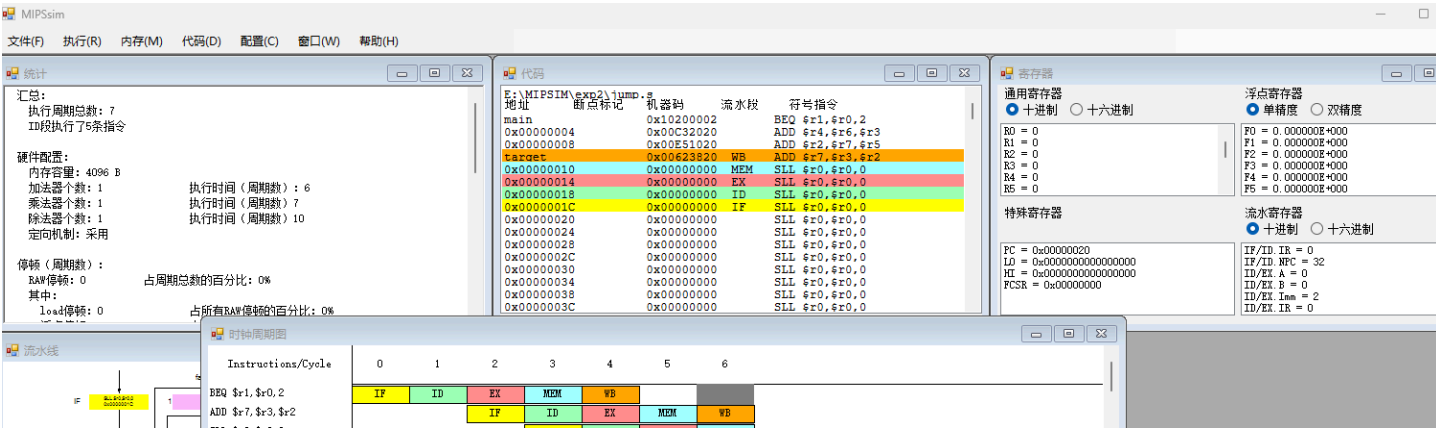
跳转场景

```
.text
main:
    BEQZ $r1, target
    ADD $r4, $r6, $r3
    ADD $r2, $r7, $r5
target:
    ADD $r7, $r3, $r2
.data
    .align 2
```

自设计模拟器运行结果



流水线模拟器运行结果



发现模拟正确。场景分析：检测到beqz指令，停止IF段流入，等待EX段检测结果。发现\$1为0，成功跳转，将目标target处的指令推入IF段。后续正常执行。

五、实验感悟

通过本次流水线模拟器设计与实现实验，我对计算机体系结构中的流水线技术有了更深刻的理解，课堂中学习的 RAW 冲突、分支冒险等概念，在实验中通过代码级实现变得具体可感。例如，在非定向场景下，lw和add的数据冒险，需要等待写回才能继续流入ID段，而定向技术也仅能减少部分阻塞周期。这种直观的周期级时序分析，让我理解了数据前递的优化能力。通过模拟 IF、ID、EX、MEM、WB 各阶段的推进逻辑，我也深入体会到流水线“重叠执行”的原理，看似并行，实则通过精细的时序控制和数据传递实现效率提升。

附录

```
import tkinter as tk # 导入tkinter库，用于GUI界面
from tkinter import ttk, messagebox, scrolledtext # 导入tkinter的子模块

REG_NAMES = [f"${i}" for i in range(32)] # 32个寄存器的名字列表

class Instruction:
    def __init__(self, text, idx):
        self.text = text.strip() # 指令文本内容
        self.idx = idx # 指令在程序中的索引
        self.stage_history = [] # 记录每周期所处阶段
        self.parse() # 解析指令
        self.finished = False # 指令是否已完成
        self.entered_if = False # 是否已进入IF阶段

    def parse(self):
        if self.text.endswith(':'):
            self.is_label = True
            self.op = "label" # 标签
            self.args = []
        else:
            self.is_label = False
            parts = self.text.replace(',', '').split() # 按空格分割，去掉逗号
            self.op = parts[0] # 操作码
            self.args = parts[1:] # 操作数参数

class RegisterFile:
    def __init__(self):
        self.regs = [0] * 32 # 32个寄存器，初始值为0

    def __getitem__(self, reg):
        idx = int(reg.replace('$', '')) # 获取寄存器编号
        return self.regs[idx] # 返回寄存器值

    def __setitem__(self, reg, value):
        idx = int(reg.replace('$', '')) # 获取寄存器编号
        self.regs[idx] = value # 设置寄存器值

class Memory:
    def __init__(self):
        self.mem = {} # 内存用字典表示，地址为key，值为value

    def __getitem__(self, addr):
        return self.mem.get(addr, 0) # 读取内存，默认值为0
```

```
def __setitem__(self, addr, value):
    self.mem[addr] = value # 写内存
```

```
class Pipeline:
```

```
    def __init__(self, regfile, memory, use_forwarding):
        self.regfile = regfile # 寄存器文件对象
        self.memory = memory # 内存对象
        self.use_forwarding = use_forwarding # 是否启用数据前递
        self.instructions = [] # 指令列表
        self.pc = 0 # 程序计数器
        self.cycle = 0 # 当前周期数
        self.finished = False # 是否执行完毕
        self.stages = [None] * 5 # 五个流水线阶段: [IF, ID, EX, MEM, WB]
        self.in_pipeline = [] # 只记录已进流水线的指令
        self.branch_pending = False # 是否有分支待处理
        self.branch_target = None # 分支目标指令索引
        self.stall = 0 # 暂停周期数 (未实际使用)
```

```
    def load_program(self, instructions):
        self.instructions = instructions # 加载指令
        self.pc = 0 # 重置PC
        self.cycle = 0 # 重置周期
        self.finished = False # 重置完成标志
        self.stages = [None] * 5 # 清空流水线
        self.in_pipeline = [] # 清空流水线指令
        self.branch_pending = False # 清空分支标志
        self.branch_target = None # 清空分支目标
        self.stall = 0 # 清空暂停
        for inst in self.instructions:
            inst.stage_history = [] # 清空每条指令的阶段历史
            inst.finished = False # 重置完成标志
            inst.entered_if = False # 重置IF标志
```

```
    def step(self):
```

```
        if self.finished:
            return
```

```
        # 检查ID段RAW冒险
```

```
        stall_needed = False # 是否需要暂停
```

```
        if not self.use_forwarding and self.stages[1]:
```

```
            id_inst = self.stages[1] # 当前ID段指令
```

```
            id_srcs = [] # ID段源操作数
```

```
            if id_inst.op == "add":
```

```

        id_srcs = [id_inst.args[1], id_inst.args[2]] # add的两个源寄存器
    elif id_inst.op == "lw":
        id_srcs = [id_inst.args[1].split('(')[1][: -1]] # lw的源寄存器
    elif id_inst.op == "sw":
        id_srcs = [id_inst.args[0], id_inst.args[1].split('(')[1][: -1]] # sw的源寄存器
    elif id_inst.op == "beqz":
        id_srcs = [id_inst.args[0]] # beqz的源寄存器
    for stage_idx in [2, 3, 4]: # EX, MEM, WB
        inst = self.stages[stage_idx] # 检查流水线后段指令
        if inst and inst.op in ("add", "lw"):
            dest = inst.args[0] # 目的寄存器
            if dest in id_srcs:
                if stage_idx != 4:
                    stall_needed = True # 检测到冒险需暂停
                    break

forwarding_stall_needed = False
if self.use_forwarding and self.stages[1]:
    ex_inst = self.stages[1]
    ex_srcs = []
    if ex_inst.op == "add":
        ex_srcs = [ex_inst.args[1], ex_inst.args[2]] # add的两个源寄存器
    elif ex_inst.op == "lw":
        ex_srcs = [ex_inst.args[1].split('(')[1][: -1]] # lw的源寄存器
    elif ex_inst.op == "sw":
        ex_srcs = [ex_inst.args[0], ex_inst.args[1].split('(')[1][: -1]] # sw的源寄存器
    elif ex_inst.op == "beqz":
        ex_srcs = [ex_inst.args[0]] # beqz的源寄存器
    inst = self.stages[2]
    if inst and inst.op == "lw":
        dest = inst.args[0]
        if dest in ex_srcs:
            forwarding_stall_needed = True

beqz_IF_pause = False
if self.stages[0] and self.stages[0].op == "beqz":
    beqz_IF_pause = True

# 检查分支冒险 (beqz在EX段判断)
if self.stages[1] and self.stages[1].op == "beqz":
    rs, label = self.stages[1].args # beqz的源寄存器和目标标签
    if self.regfile[rs] == 0:
        for i, ins in enumerate(self.instructions):
            if ins.text.startswith(label + ":"):
                self.branch_pending = True # 标记分支待处理

```

```

        self.branch_target = i+1 # 记录分支目标
        break
    else:
        self.branch_pending = False # 不跳转

# 推进流水线（后段先推进）
self.stages[4] = self.stages[3] # MEM->WB
self.stages[3] = self.stages[2] # EX->MEM

if self.use_forwarding and self.stages[1]:
    if forwarding_stall_needed:
        self.stages[1].stage_history.append('ID')
    else:
        if self.stages[1].stage_history[-1] == 'ID':
            self.stages[1].stage_history.append('stall')
        else:
            self.stages[1].stage_history.append('ID')

# 定向技术处理ID/EX推进和写入
if forwarding_stall_needed:
    self.stages[2] = None

    if self.stages[0] and self.stages[0].entered_if:
        self.stages[0].stage_history.append('stall')
# 非定向技术处理ID/EX推进和写入
elif stall_needed:

    # EX段不推进，ID段写入stall
    self.stages[2] = None
    if self.stages[1]:
        self.stages[1].stage_history.append('stall') # 记录暂停
    # IF段不推进，已进入IF的指令补stall
    if self.stages[0] and self.stages[0].entered_if:
        self.stages[0].stage_history.append('stall')
else:

    # EX段推进
    self.stages[2] = self.stages[1]
    # ID段推进，写入ID
    if self.stages[1] and not self.use_forwarding:
        self.stages[1].stage_history.append('ID') # 记录ID阶段
    self.stages[1] = self.stages[0]
    # IF段推进
    if self.branch_pending:
        if self.branch_target is not None:
            inst = self.instructions[self.branch_target] # 分支目标指令

```

```

        while len(inst.stage_history) < self.cycle:
            inst.stage_history.append("") # 补齐历史
        if not inst.entered_if:
            inst.stage_history.append('IF') # 记录IF阶段
            inst.entered_if = True
            self.in_pipeline.append(inst) # 加入流水线
            self.stages[0] = inst # IF段为分支目标
            self.pc = self.branch_target + 1 # PC跳转
        else:
            self.stages[0] = None
        self.branch_pending = False # 清空分支标志
        self.branch_target = None
    else:
        if self.pc < len(self.instructions) and not beqz_IF_pause:

            inst = self.instructions[self.pc] # 取下一条指令
            while inst.is_label:
                self.pc += 1
                inst = self.instructions[self.pc]
            while len(inst.stage_history) < self.cycle:
                inst.stage_history.append("")
            if not inst.entered_if:
                inst.stage_history.append('IF')
                inst.entered_if = True
                self.in_pipeline.append(inst)
            self.stages[0] = inst
            self.pc += 1 # PC递增
        else:
            self.stages[0] = None

# 其它流水线阶段推进和写入
# EX段
if self.stages[2]:
    self.stages[2].stage_history.append('EX') # 记录EX阶段
# MEM段
if self.stages[3]:
    self.stages[3].stage_history.append('MEM') # 记录MEM阶段
# WB段
if self.stages[4] and not self.stages[4].finished:
    self.stages[4].stage_history.append('WB') # 记录WB阶段
    self.write_back(self.stages[4]) # 写回
    self.stages[4].finished = True # 标记完成

# MEM/EX段操作
if self.stages[3]:
    self.memory_access(self.stages[3]) # 访存

```

```

if self.stages[2]:
    self.execute(self.stages[2]) # 执行

self.cycle += 1 # 周期加一

# 检查是否全部完成
if all(x is None or getattr(x, 'finished', False) for x in self.stages) and self.pc > 1000:
    self.finished = True
if all(x is None or getattr(x, 'finished', False) for x in self.stages) and self.pc >= 1000:
    self.pc += 1

def execute(self, inst):
    # 执行阶段: 计算结果或地址
    if inst.op == "add":
        rd, rs, rt = inst.args # 目的寄存器、两个源寄存器
        inst.result = self.regfile[rs] + self.regfile[rt] # 计算加法
    elif inst.op == "lw":
        rt, offset_rs = inst.args # 目标寄存器, 偏移量和基址
        offset, rs = offset_rs.replace(' ', '').split('(')
        addr = self.regfile[rs] + int(offset) # 计算内存地址
        inst.mem_addr = addr # 保存地址
    elif inst.op == "sw":
        rt, offset_rs = inst.args # 源寄存器, 偏移量和基址
        offset, rs = offset_rs.replace(' ', '').split('(')
        addr = self.regfile[rs] + int(offset)
        inst.mem_addr = addr
    # beqz在step里处理分支冒险

def memory_access(self, inst):
    # 访存阶段
    if inst.op == "lw":
        inst.result = self.memory[inst.mem_addr] # 读取内存
    elif inst.op == "sw":
        self.memory[inst.mem_addr] = self.regfile[inst.args[0]] # 写内存

def write_back(self, inst):
    # 写回阶段
    if inst.op == "add":
        self.regfile[inst.args[0]] = inst.result # 写回寄存器
    elif inst.op == "lw":
        self.regfile[inst.args[0]] = inst.result # 写回寄存器

class PipelineGUI:
    def __init__(self, root):

```



```

self.root = root # 主窗口
self.root.title("MIPS五段流水线模拟器") # 窗口标题

self.instructions = [] # 指令列表
self.regfile = RegisterFile() # 寄存器文件
self.memory = Memory() # 内存
self.pipeline = Pipeline(self.regfile, self.memory, use_forwarding=False) # 流水线对象

self.code_input = scrolledtext.ScrolledText(root, width=40, height=10) # 代码输入框
self.code_input.grid(row=0, column=0, columnspan=4, padx=5, pady=5)
self.load_btn = tk.Button(root, text="加载程序", command=self.load_program) # 加载按钮
self.load_btn.grid(row=1, column=0)
self.step_btn = tk.Button(root, text="单步执行", command=self.step) # 单步执行按钮
self.step_btn.grid(row=1, column=1)
self.run_btn = tk.Button(root, text="运行到结束", command=self.run_to_end) # 运行到结束
self.run_btn.grid(row=1, column=2)
self.forward_var = tk.BooleanVar() # 是否启用前递的变量
self.forward_check = tk.Checkbutton(root, text="定向(数据前递)", variable=self.forward_
self.forward_check.grid(row=1, column=3)

self.pipeline_table = ttk.Treeview(root, columns=[], show="headings", height=10)
self.pipeline_table.grid(row=2, column=0, columnspan=4, padx=5, pady=5, sticky="nsew")

# 新增：横向滚动条
self.pipeline_xscroll = ttk.Scrollbar(root, orient="horizontal", command=self.pipeline
self.pipeline_table.configure(xscrollcommand=self.pipeline_xscroll.set)
self.pipeline_xscroll.grid(row=3, column=0, columnspan=4, sticky="ew")

# 让主窗口支持自动拉伸
self.root.grid_rowconfigure(2, weight=1)
self.root.grid_columnconfigure(0, weight=1)
self.root.grid_columnconfigure(1, weight=1)
self.root.grid_columnconfigure(2, weight=1)
self.root.grid_columnconfigure(3, weight=1)

self.reg_table = ttk.Treeview(root, columns=["寄存器", "值"], show="headings", height=8)
self.reg_table.heading("寄存器", text="寄存器")
self.reg_table.heading("值", text="值")
self.reg_table.grid(row=4, column=0, columnspan=2, padx=5, pady=5)

self.status_label = tk.Label(root, text="状态：等待加载程序") # 状态标签
self.status_label.grid(row=5, column=0, columnspan=4)

self.update_reg_table() # 初始化寄存器表

def load_program(self):

```

```

code = self.code_input.get("1.0", tk.END).strip().split('\n') # 获取输入的代码
self.instructions = [Instruction(line, idx) for idx, line in enumerate(code) if line.s
self.regfile = RegisterFile() # 重置寄存器
self.memory = Memory() # 重置内存
self.pipeline = Pipeline(self.regfile, self.memory, self.forward_var.get()) # 新建流水
self.pipeline.load_program(self.instructions) # 加载指令
self.status_label.config(text="状态: 已加载, 等待执行") # 更新状态
self.update_pipeline_table() # 刷新流水线表
self.update_reg_table() # 刷新寄存器表

def step(self):
    if not self.pipeline or not self.instructions:
        messagebox.showwarning("警告", "请先加载程序") # 未加载程序警告
        return
    if self.pipeline.finished:
        self.status_label.config(text="状态: 程序已结束") # 程序结束
        return
    self.pipeline.step() # 执行一步
    self.update_pipeline_table() # 刷新流水线表
    self.update_reg_table() # 刷新寄存器表
    self.status_label.config(text=f"状态: 已执行{self.pipeline.cycle}周期") # 更新状态

def run_to_end(self):
    if not self.pipeline or not self.instructions:
        messagebox.showwarning("警告", "请先加载程序") # 未加载程序警告
        return
    while not self.pipeline.finished:
        self.pipeline.step() # 一直执行到结束
    self.update_pipeline_table() # 刷新流水线表
    self.update_reg_table() # 刷新寄存器表
    self.status_label.config(text=f"状态: 程序已结束, 共{self.pipeline.cycle}周期") # 更新状

def update_pipeline_table(self):
    self.pipeline_table.delete(*self.pipeline_table.get_children()) # 清空表格
    in_pipeline = self.pipeline.in_pipeline if self.pipeline else [] # 当前流水线中的指令
    if not in_pipeline:
        return
    # 只显示到 self.pipeline.cycle-1 列
    max_cycle = max((len(inst.stage_history) for inst in in_pipeline), default=0) # 最大周
    show_cycle = max(0, self.pipeline.cycle - 1) # 当前显示周期数
    columns = ["指令"] + [f"C{c+1}" for c in range(show_cycle)] # 列名
    self.pipeline_table["columns"] = columns
    for idx, col in enumerate(columns):
        self.pipeline_table.heading(col, text=col)
        if idx == 0:
            # 第一列"指令"宽度设为200像素

```

```

        self.pipeline_table.column(col, width=200, minwidth=200, stretch=False)
    else:
        # 其它列宽度设为40像素
        self.pipeline_table.column(col, width=40, minwidth=40, stretch=False)
    for inst in in_pipeline:
        row = [inst.text] + inst.stage_history[:show_cycle] + [""] * (show_cycle - len(inst.stage_history))
        self.pipeline_table.insert("", "end", values=row)

def update_reg_table(self):
    self.reg_table.delete(*self.reg_table.get_children()) # 清空寄存器表
    for i in range(32):
        self.reg_table.insert("", "end", values=(f"${i}", self.regfile.regs[i])) # 插入寄存器值

def toggle_forwarding(self):
    if self.pipeline:
        self.pipeline.use_forwarding = self.forward_var.get() # 切换前递模式

if __name__ == "__main__":
    root = tk.Tk() # 创建主窗口
    app = PipelineGUI(root) # 创建GUI应用
    root.mainloop() # 进入主循环

```